

DEEP LEARNING, TOLD SIMPLY

Book 3 of 6

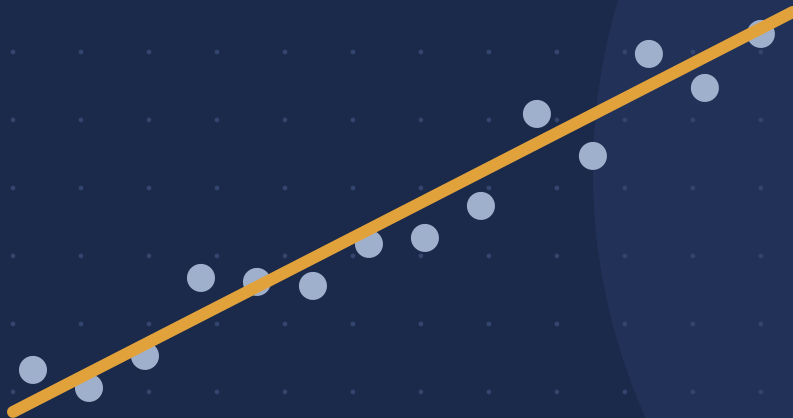
WRITTEN BY

Prof. Saurabh

Guessing How Much

A single neuron that draws a straight line, a loss shaped like a bowl, and the walk downhill called gradient descent

Meena aunty sells flats. She asks Nova a simple question: given the size of a flat, how much will it sell for? Asha discovers that 'how much' questions have a name — regression — and that one neuron, one loss and one learning rule are enough to answer them. This book follows Module 3 of the Deep Neural Network course, slide by slide, with every number worked out by hand.



What is inside

Book 3 covers Module 3: Linear Neural Networks for Regression. Seven short chapters. The first four build the machine; chapter 5 runs it on the three flats from your slides, number by number; the last two tell you how to judge the result and how to fix it when it goes wrong.

1	"How much?" — what regression is	3
	When the answer is a number on a line, not a name from a list, the problem is called regression. Linear regression answers it with a straight line.	
2	Four parts, and one neuron that draws a line	7
	The same four parts as always: data in a matrix, a model that is a single neuron, a loss, and a learning rule. This chapter builds the first two.	
3	How wrong is the line?	11
	One number, the loss, says how bad all the guesses are. Drawn over all possible weights, it is a bowl with one lowest point.	
4	Walking downhill: gradient descent	15
	Stand on the bowl, feel which way is steepest, take a small step the other way. Repeat until the ground is flat. That is the whole learning algorithm.	
5	Nova's first price guess, number by number	19
	Three flats, two weights, one learning step, every multiplication written out. Then ten more steps in a table, and the line Nova draws after each.	
6	Was it a good guess? Evaluation	23
	A model is judged on flats it has never seen. Four numbers tell you how good it is: MSE, RMSE, MAE and R^2 .	
7	Tips, debugging, and the big picture	27
	How to pick the learning rate, when to stop, why to scale your features, what to check when the loss misbehaves, and the one picture that holds the whole module.	

How to read this book

Each chapter follows the same rhythm: a little story, one big idea, an everyday picture of it, the maths in small steps (first plain, then in matrix form), a worked example with real numbers, and a short check. The coloured boxes tell you what kind of thing you are reading:

- | | |
|--|--|
|  Story — Asha and Nova, the thread through all six books |  Big idea — the one thing to remember |
|  Think of it like... — a real-life picture of the idea |  Stop and think — a question, then Our answer |
|  Maths, step by step — plain maths → matrix form |  Worked example — real numbers, one line per step |
|  On your slides — the exact notation your deck uses |  Careful — common mistakes |
|  Spark — a deeper idea for the curious |  Check yourself — quick questions with answers |

"How much?" — what regression is

When the answer is a number on a line, not a name from a list, the problem is called regression. Linear regression answers it with a straight line.

On your slides: **Module 3, slides 4–9** · What is regression · Examples · What is linear regression · Examples · 2-D visual example

📖 STORY

Meena aunty sells flats. On Sunday she visits Asha's hostel with a notebook: the size of every flat she has sold, and the price it went for. "Your robot is clever, no?" she says. "Tell me what this new flat will sell for. It is 1,750 square feet." Asha looks at Nova. Nova can only say yes or no. A price is a number, and it could be any number. "This is a different kind of question," Asha says. Then she notices something. Nova also needs a number to decide *how loud* to reply. Two "how much" questions in one evening.

! BIG IDEA

A **regression** problem asks "how much?" The answer is a real number, like a price. **Linear regression** is the simplest way to answer: multiply each input by a weight, add them up, add a starting value. The result is a straight line through the data.

≈ THINK OF IT LIKE...

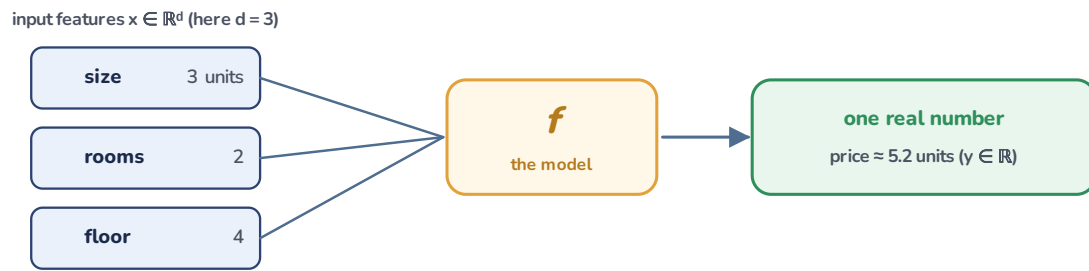
An auto-rickshaw meter. The fare starts at a fixed amount before you move; then every kilometre adds a fixed amount. $\text{Fare} = \text{starting amount} + \text{rate} \times \text{kilometres}$. That is a straight line. The starting amount is the *bias*; the rate per kilometre is the *weight* of the input "kilometres". Linear regression is a meter with one rate for every input.

What regression is

Your slides define it in one sentence. **Regression is a supervised learning task that predicts a continuous-valued output based on input features.** Word by word:

- *Supervised*: every example comes with the right answer (Book 1). Meena aunty's notebook has the size *and* the price of each flat.
- *Continuous-valued output*: the answer can be any number on the number line, like 42.5 or 43.1. Not just "yes" or "no".
- *Input features*: the numbers we know about each example. Size, number of rooms, floor.

Ravi asks whether "is that my name?" is regression. No: its answer is yes or no, a choice, and that is classification (Book 4). But "how loud should I reply?" has a number as its answer, 3 out of 10 or 7.5 out of 10. That is regression. Nova needs both.



Regression: many numbers in, one number out. The number can be anything on the number line.

Figure 1.1 Regression as a box. Several numbers go in (the features). One number comes out (the prediction). Notice that the output is a point on a line, not a choice from a list. That is the whole difference from Book 2's yes/no neuron.

Σ MATHS, STEP BY STEP

PLAIN MATHS

The input is a list of d numbers. We write it as a vector $\mathbf{x}$ and say $\mathbf{x} \in \mathbb{R}^d$. The symbol $\mathbb{R}^d$ just means "a list of d real numbers".

The output is one real number: $y \in \mathbb{R}$.

The goal is to learn a rule f that turns the list into the number:

$$f : \mathbb{R}^d \rightarrow \mathbb{R}, \quad y \approx f(\mathbf{x})$$

The wavy equals sign $\approx$ means "close to". We do not expect a perfect answer, only a good one.

MATRIX FORM

With N examples we stack the inputs as rows of a table $\mathbf{X}$. The answers go in one column, $\mathbf{y}$.

$$\mathbf{X} \in \mathbb{R}^{N \times d}, \quad \mathbf{y} \in \mathbb{R}^N$$

The rule f is then applied to every row at once, giving a column of N guesses. Chapter 2 shows exactly how.

THREE EXAMPLES FROM YOUR SLIDES

Problem	Input features	Output (a number)
House price prediction	size, location, number of rooms	price in rupees or dollars
Temperature forecasting	past weather, season, humidity	temperature in degrees
Stock price prediction	past prices, trading volume, market indicators	the future price

All three share one shape: a few numbers in, one number out. So one method can handle all of them.

What linear regression is

Regression says *what* we want: a number. Linear regression says *how* we will get it. Your slides define it like this: **linear regression assumes the output is a linear combination of the input features**. A **linear combination** is the simplest recipe there is: multiply each input by its own number, and add everything up. Nothing is squared, nothing is multiplied by another input.

THREE LINEAR REGRESSIONS FROM YOUR SLIDES

- **Salary prediction:** $\text{Salary} = w_0 + w_1 \times \text{Years of experience}$. One feature, one weight.
- **Crop yield:** $\text{Yield} = w_0 + w_1 \times \text{Rainfall} + w_2 \times \text{Fertilizer}$. Two features, two weights.
- **Energy use:** $\text{Energy} = w_0 + w_1 \times \text{Temperature} + w_2 \times \text{Hour} + w_3 \times \text{Day}$. Three features, three weights.

PLAIN MATHS

With d input features, the model is

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

Here $x_1, \dots, x_d$ are the input features. Each w_j is a number that says how much feature j matters; it is called a **weight**. The extra number w_0 is added no matter what the inputs are; it is called the **bias**. Your slides write the inputs as $\mathbf{x} = [x_1, \dots, x_d]^\top$ and the weights as $\mathbf{w} = [w_1, \dots, w_d]^\top$.

With one feature (size only) this is just $y = w_0 + w_1x_1$: a straight line.

MATRIX FORM

Write the multiply-and-add as one dot product:

$$y = w_0 + \mathbf{w}^\top \mathbf{x}$$

$\mathbf{w}^\top \mathbf{x}$ means "multiply matching entries and add": $w_1x_1 + w_2x_2 + \dots + w_dx_d$. The bias w_0 still sits outside. In Chapter 2 we tuck it inside too, by giving every example a fake first feature equal to 1.

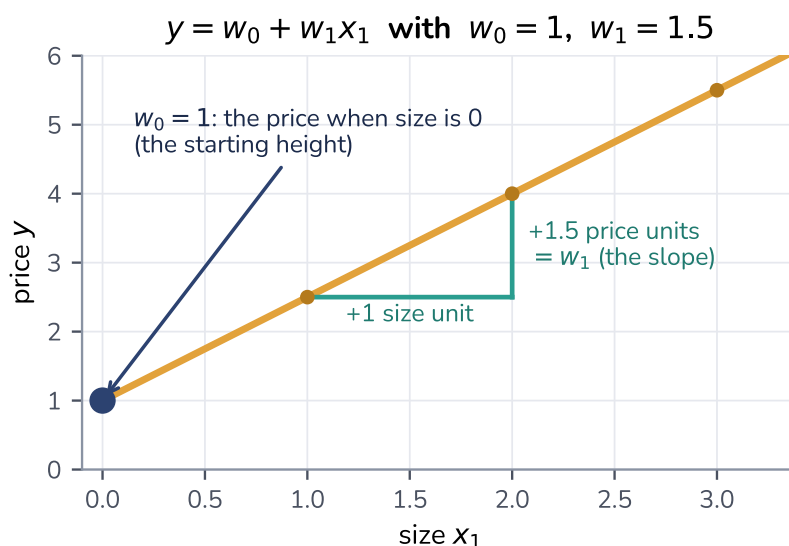


Figure 1.2 What the two numbers mean on a picture, for one feature. w_0 is the height of the line at size 0: the "starting fare". w_1 is the slope: how much the price rises when the size goes up by one unit. Change w_0 and the line moves up or down. Change w_1 and it tilts.

Each feature gets one weight; the bias is added at the end. The general form is in the ladder above. Your slide ends with the key assumption: *the relationship between inputs and output is assumed to be linear*. If the real world bends, a straight line is a rough guess. We accept that for now.

The picture: a cloud of dots and one line

Here is the picture your slide 9 shows, drawn with Meena aunty's flats. Every dot is one flat she has sold. Left-right is the size. Up-down is the price. The dots do not sit on a perfect line, because real prices are messy. But they clearly rise together. Linear regression draws the one straight line that sits closest to all the dots.



Figure 1.3 Two-dimensional linear regression. Each dot is a flat: its size and its price. The orange line is the model. Notice how you use it. Go up from a new size (the dotted line at 3.5) until you hit the orange line. Then read the price off the left axis. The dashed grey line is the "true" rule that produced the data; the fitted line is close to it. Your slide shows the same picture with years of experience and salary. Units in this book: one size unit is 500 square feet, one price unit is ten lakh rupees. The maths does not care about units; only the story does.

ON YOUR SLIDES

Slide 4: regression, with $\mathbf{x} \in \mathbb{R}^d$, $y \in \mathbb{R}$, goal $f : \mathbb{R}^d \rightarrow \mathbb{R}$, $y \approx f(\mathbf{x})$. Slide 5: the three examples. Slide 7: equation (1), $y = w_0 + w_1x_1 + \dots + w_dx_d$, with $\mathbf{x}$ the input features, $\mathbf{w}$ the weights, w_0 the bias term. Slide 8: salary, crop yield, energy. Slide 9: the scatter plot with the fitted line.

CAREFUL

"Linear" is about the *weights*, not about the picture being a line. With two features the model is a flat sheet in three dimensions; with d features, a flat thing in $d + 1$ dimensions that nobody can draw. The rule is the same: multiply, add, done.

CHECK YOURSELF

1. Predicting tomorrow's temperature: regression or classification? **Regression. The answer is a number on a line.**
2. In $y = w_0 + w_1x_1$, which number moves the line up and down? **w_0 , the bias.**
3. What does $f : \mathbb{R}^d \rightarrow \mathbb{R}$ say in words? **A rule that takes a list of d numbers and gives back one number.**
4. How many weights (not counting the bias) does the energy example use? **Three: one each for temperature, hour and day.**

IN ONE BREATH

- Regression = "how much?" The output is a real number, $y \in \mathbb{R}$.
- Linear regression: $y = w_0 + w_1x_1 + \dots + w_dx_d$. Multiply each feature by a weight, add the bias.
- One feature gives a line: w_0 is the height at zero, w_1 the slope. It assumes the world is straight.

Four parts, and one neuron that draws a line

The same four parts as always: data in a matrix, a model that is a single neuron, a loss, and a learning rule. This chapter builds the first two.

On your slides: **Module 3, slides 11–16** · The four components · Matrix form of data · Linear model as a single neuron · Prediction · Identity activation

STORY

Asha copies Meena aunty's numbers into a table: one row per flat, size in one column, price in the next. "Nova needs this as a matrix," she says. Ravi leans in the doorway. "And then what? Nova's neuron only knows how to say yes or no." Asha smiles. "Same neuron. I just take off the last step." She opens Book 2 to the neuron picture and crosses out the step function.

BIG IDEA

Linear regression is **one artificial neuron**. It multiplies each input by a weight, adds the bias, and passes the sum through the **identity** function, which changes nothing. So the output is the weighted sum itself: any real number. With all the examples stacked in a matrix $\mathbf{X}$, all the predictions at once are $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$.

THINK OF IT LIKE...

A shop bill. Each item has a quantity (the input) and a price per unit (the weight). The cashier multiplies and adds, then adds a fixed packing charge (the bias). Nobody rounds the bill to "yes" or "no": the total is just a number. That cashier is our neuron. With a hundred customers you do not add up a hundred bills one at a time. You put all the quantities in a table and multiply the whole table by the price list in one go. That is the matrix form.

The four parts, for regression

Book 1 said every learning problem has four parts. Your slide 11 fills them in for linear regression.

1. Data

d -dimensional input vectors, each with a target number.

2. Model

A single neuron that computes a linear function.

3. Objective function

Measures the prediction error. Lower is better.

4. Learning algorithm

Gradient descent, to find good weights.

This chapter does parts 1 and 2. Chapter 3 does part 3. Chapter 4 does part 4.

Part 1: the data as a matrix

Computers are fast at one thing: multiplying big tables of numbers. So we arrange the data as tables. Your slides call this the **matrix form of data**.

First, a small trick. The model is $y = w_0 + w_1x_1 + \dots + w_dx_d$. The bias w_0 is the odd one out: it has no input to multiply. So we give it one. We add a fake feature that is always equal to 1 at the front of every example. Then $w_0 \times 1 = w_0$, and the bias behaves like every other weight. Your slides call this the **augmented** input and write it with a tilde: $\tilde{\mathbf{x}}$.

Σ MATHS, STEP BY STEP

PLAIN MATHS

One example, with the 1 added in front:

$$\tilde{\mathbf{x}}^{(i)} = [1, x_1^{(i)}, \dots, x_d^{(i)}]^\top$$

The little (i) means "example number i ". The weights, bias first:

$$\mathbf{w} = [w_0, w_1, \dots, w_d]^\top$$

Both lists now have $d + 1$ entries. Multiply matching entries and add:

$$\mathbf{w}^\top \tilde{\mathbf{x}}^{(i)} = w_0 \cdot 1 + w_1x_1^{(i)} + \dots + w_dx_d^{(i)}$$

That is exactly the model from Chapter 1.

The bias is inside now.

MATRIX FORM

Stack the N augmented examples as rows.

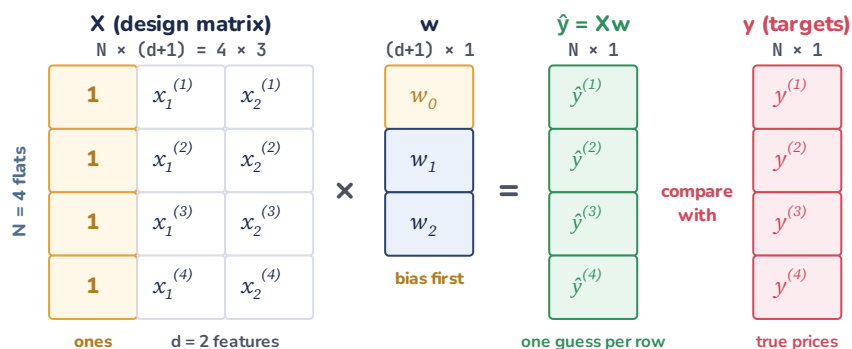
This is the **design matrix**:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix} \in \mathbb{R}^{N \times (d+1)}$$

The targets go in one column, and the weights in one column:

$$\mathbf{y} = [y^{(1)}, \dots, y^{(N)}]^\top \in \mathbb{R}^N, \quad \mathbf{w} \in \mathbb{R}^{d+1}$$

Read the shapes aloud: $\mathbf{X}$ has N rows (one per example) and $d + 1$ columns (the 1, then the d features).



Row i of $\mathbf{X}$ times the column $\mathbf{w}$ gives $\hat{y}^{(i)} = w_0 \cdot 1 + w_1x_1^{(i)} + w_2x_2^{(i)}$. The ones column is what multiplies the bias.

Figure 2.1 The shapes, for $N = 4$ flats and $d = 2$ features. $\mathbf{X}$ is 4×3 ; its first column is all ones (cream). $\mathbf{w}$ is 3×1 , bias first. Multiplying gives $\hat{\mathbf{y}}$, a 4×1 column with one guess per flat. Notice that the number of columns of $\mathbf{X}$ (3) matches the number of rows of $\mathbf{w}$ (3). That match is what makes the multiplication possible.

Part 2: the model is one neuron

Here is the neuron from Book 2 again. Inputs on the left, a weight on every wire, a sum in the middle, then an activation function f , then the output $\hat{y}$. The hat on $\hat{y}$ means "our guess", to keep it apart from the true y .

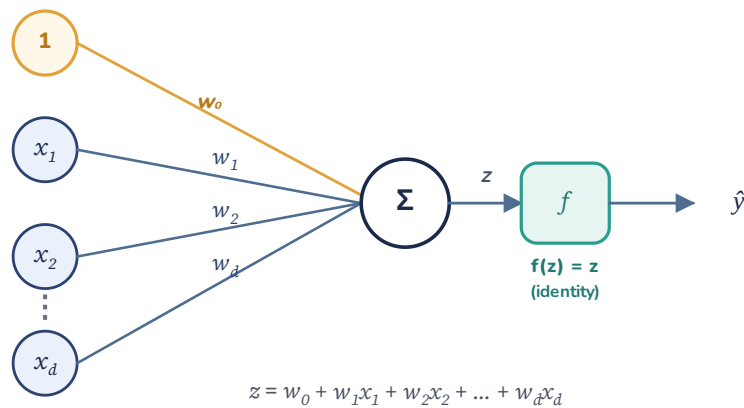


Figure 2.2 Linear regression as a single neuron. The bias is a weight w_0 on a constant input of 1 (cream circle). Σ adds up $w_0 + w_1x_1 + \dots + w_dx_d$ and calls it z . The box f is the activation; for regression it is the identity, so $\hat{y}$ is simply z .

Σ MATHS, STEP BY STEP

PLAIN MATHS

For one input $\mathbf{x}$ (with the 1 in front), the neuron computes

$$\hat{y} = f(\mathbf{w}^\top \mathbf{x}) = f(w_0 + w_1x_1 + \dots + w_dx_d)$$

f is the identity, $f(z) = z$. So the f does nothing and

$$\hat{y} = w_0 + w_1x_1 + \dots + w_dx_d$$

Input: a d -dimensional feature vector.

Output: a single number $\hat{y} \in \mathbb{R}$.

MATRIX FORM

All N predictions in one line:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

Shapes: $\mathbf{X}$ is $N \times (d+1)$, $\mathbf{w}$ is $(d+1) \times 1$, so $\mathbf{X}\mathbf{w}$ is $N \times 1$. Row i of $\mathbf{X}$ times $\mathbf{w}$ is exactly $\mathbf{w}^\top \tilde{\mathbf{x}}^{(i)}$, the plain version for example i .

This is what code actually runs: one matrix multiplication instead of a loop over N examples. On a modern chip that is hundreds of times faster.

The identity activation

Book 2's neuron ended with a step: output 1 if the sum was big enough, else 0. That was right for a yes/no decision. For a price we want the sum itself. So we use the simplest activation function there is: the **identity**, $f(z) = z$. Whatever goes in comes out unchanged.

PROPERTIES, AND WHY THEY SUIT REGRESSION

- **Output is continuous**, anywhere in $(-\infty, \infty)$: any real number, not just $\{0, 1\}$.
- **Differentiable everywhere**, with slope $f'(z) = 1$ (a derivative is just a slope). This lets gradient-based learning (Chapter 4) work.
- **The gradient flows unchanged** through the neuron: multiplying by 1 changes nothing.
- **No information is lost**: what the sum says, the neuron says.

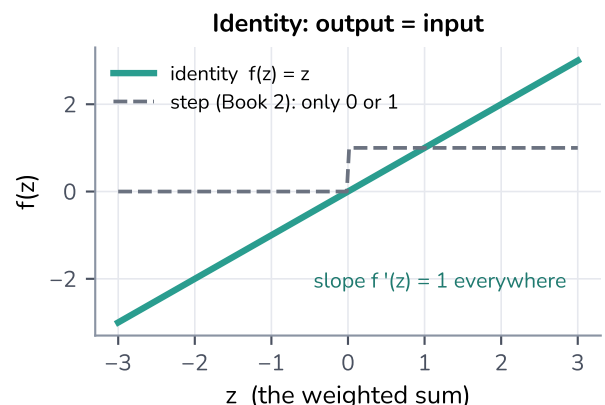


Figure 2.3 The identity is the diagonal $f(z) = z$; the step from Book 2 (dashed) only gives 0 or 1. Notice: no flat part, no corner, slope 1 everywhere, so the gradient never gets stuck.

? STOP AND THINK

Ravi: "If the activation does nothing, why draw it at all?"

✓ OUR ANSWER

Because it is a slot. In Book 4 we will put a sigmoid in that slot, and the same neuron will output a probability. In Book 5 we will put a ReLU there. Drawing the empty slot now means the picture never has to change later. Only what is inside f changes.

📖 ON YOUR SLIDES

Slide 11: the four components. Slide 12: $\tilde{\mathbf{x}}^{(i)} = [1, x_1^{(i)}, \dots, x_d^{(i)}]^\top$, design matrix $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$, $\mathbf{y} \in \mathbb{R}^N$, $\mathbf{w} \in \mathbb{R}^{d+1}$. Slide 13: the neuron picture. Slide 14: equation (2), $\hat{y} = f(\mathbf{w}^\top \mathbf{x})$, and equation (3), $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. Slides 15–16: equation (4), $f(z) = z$, its properties, and why identity for regression.

! CAREFUL

Two things are easy to mix up. (1) The *ones column* is part of $\mathbf{X}$, so $\mathbf{X}$ has $d + 1$ columns, not d . (2) In this deck the bias is called w_0 and lives inside $\mathbf{w}$. Book 2 called it b and kept it separate. Same number, different name.

✓ CHECK YOURSELF

1. Why add a column of ones to $\mathbf{X}$? **So the bias w_0 can be treated as a weight on a constant input of 1.**
2. What is the shape of $\mathbf{X}$ for N examples and d features? $N \times (d + 1)$.
3. What is $\mathbf{X}\mathbf{w}$? **The column of all N predictions, $\hat{\mathbf{y}}$.**
4. What is the slope of the identity function? **1, everywhere.**
5. Why not use the step function for prices? **A step only outputs 0 or 1. A price can be any real number.**

👉 IN ONE BREATH

- Data: $\mathbf{X}$ is $N \times (d + 1)$ with a ones column in front; $\mathbf{y}$ and $\mathbf{w}$ are columns.
- Model: one neuron, $\hat{y} = f(\mathbf{w}^\top \mathbf{x})$ with $f(z) = z$, so $\hat{\mathbf{y}} = \mathbf{w}^\top \mathbf{x}$.
- All predictions at once: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. One multiplication, no loop.
- Identity activation: continuous output, slope 1, gradient passes straight through.

How wrong is the line?

One number, the loss, says how bad all the guesses are. Drawn over all possible weights, it is a bowl with one lowest point.

On your slides: **Module 3, slides 17–19** · Objective function: squared error loss · Why squared error · Vector form · Visualising the error surface

STORY

Asha tries a line by hand, and Nova's screen shows three guesses: 1.5, 2.5, 3.5. Meena aunty reads the true prices: 2, 4, 5. "All too low. By how much?" Asha subtracts: 0.5, 1.5, 1.5. "So how wrong is Nova, in total?" Ravi asks. Asha needs *one* number for "how wrong". Not three. One.

BIG IDEA

For each example, take guess minus truth, square it, halve it; average over all examples. That single number is the **squared error loss** $J(\mathbf{w})$. Plot it over all possible weights and you get a **bowl** with one lowest point and no traps. Learning means finding the bottom.

THINK OF IT LIKE...

A carrom board that is not flat but bowl-shaped. Wherever you place the striker, it rolls down to the same lowest point, with no dents on the way to get stuck in. The striker's position is the pair of weights (w_0, w_1) . The height of the board there is the loss. The lowest point is the best line.

The loss for one example

Start with a single flat. The model guesses $\hat{y}$; the truth is y ; the mistake is $\hat{y} - y$. It can be negative, and a big mistake should count much more than a small one. Squaring fixes both. The recipe for all N flats: square each mistake, add them up, divide by $2N$ (the 2 only makes the derivative tidy later). Numbers first:

WORKED EXAMPLE: THE LOSS OF ONE LINE

The three flats: sizes 1, 2, 3, prices $\mathbf{y} = (2, 4, 5)$, $N = 3$. Try the line $w_0 = 0.5$, $w_1 = 1$. Its guesses are $0.5 + 1 \cdot 1 = 1.5$, $0.5 + 1 \cdot 2 = 2.5$, $0.5 + 1 \cdot 3 = 3.5$.

- 1 Mistakes $\hat{y} - y$, one per flat: $1.5 - 2 = -0.5$; $2.5 - 4 = -1.5$; $3.5 - 5 = -1.5$.
- 2 Squares (the minus signs disappear): $(-0.5)^2 = 0.25$; $(-1.5)^2 = 2.25$; $(-1.5)^2 = 2.25$.
- 3 Sum: $0.25 + 2.25 + 2.25 = 4.75$.
- 4 $J = \frac{1}{2N} \times 4.75 = \frac{1}{6} \times 4.75 = 0.792$.
divide by $2N = 6$

One number; lower is better. Chapter 5 starts from a worse line (loss 7.5) and watches this number fall.

PLAIN MATHS

Loss for a single example i , written ℓ (small L):

$$\ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)})^2$$

The second form just replaces $\hat{y}^{(i)}$ by what the neuron computes, $\mathbf{w}^\top \mathbf{x}^{(i)}$.

Total loss = the average of the per-example losses over all N examples:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)})^2$$

Your slides call this the **mean squared error**.

Goal: the weights that make J smallest,

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w})$$

"arg min" means "the argument (the $\mathbf{w}$) at which the minimum happens".

MATRIX FORM

All the guesses at once are $\mathbf{X}\mathbf{w}$. All the mistakes at once are the column $\mathbf{X}\mathbf{w} - \mathbf{y}$. Squaring and adding every entry of a column is its squared **length**, written $\|\cdot\|^2$:

$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

The same thing written with a transpose (this is how the gradient gets derived in Chapter 4):

$$J(\mathbf{w}) = \frac{1}{2N} (\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Shapes: $\mathbf{X}\mathbf{w} - \mathbf{y}$ is $N \times 1$. A $1 \times N$ row times an $N \times 1$ column is 1×1 : a single number.

Good, a loss must be one number.

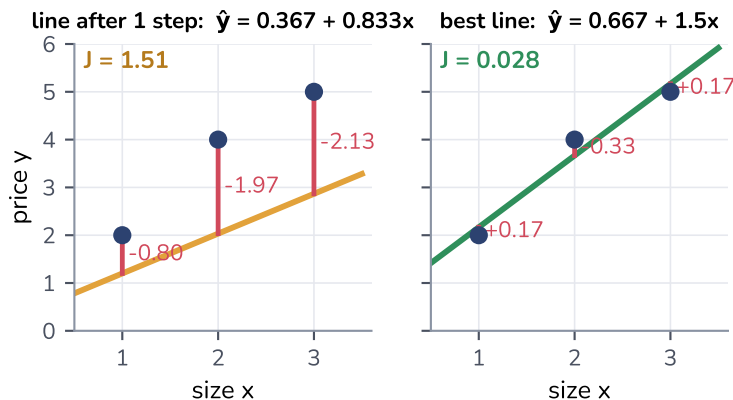


Figure 3.1 What the loss measures: the red sticks are the mistakes $\hat{y} - y$, the gap between each dot and the line. Left: a poor line (Nova's after one step in Chapter 5), $J = 1.51$. Right: the best line, $J = 0.028$. Notice that even the best line misses every dot; it balances them.

! CAREFUL

Two versions of "MSE" float around. The deck's loss J has $\frac{1}{2N}$: it halves each squared error. The evaluation metric MSE in Chapter 6 has $\frac{1}{N}$: no halving. They differ by exactly a factor of 2. The best weights are the same for both, because halving every height of a bowl does not move its bottom.

Why squared error?

Your slide 18 gives four reasons. Here they are in plain words.

- **Differentiable.** e^2 is smooth, with a slope everywhere, even at zero, so gradient descent can always tell which way is down. $|e|$ has a sharp corner at zero.
- **Convex.** For a linear model the loss is a bowl: exactly one lowest point and no local dips. Wherever you start, downhill leads to the same place.
- **Penalises large errors.** Doubling the mistake makes the cost four times bigger: a mistake of 3 costs 9. The model is pushed hard to fix its worst guesses.
- **A statistical meaning.** Suppose the true prices are "a straight line plus random noise that follows the bell curve". Then the line with the smallest squared error is the most likely line to have produced the data. Your slides say it in five words: *maximum likelihood under Gaussian noise*. No proof needed; just know squared error is not an arbitrary choice.

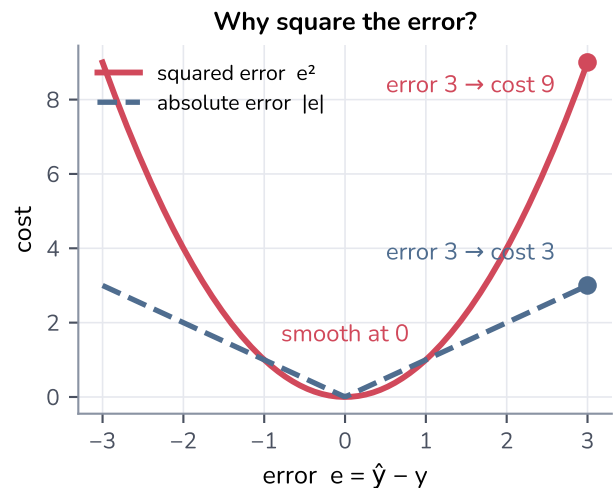


Figure 3.2 Squared error against absolute error. Notice how fast the red curve climbs: an error of 3 costs 9, not 3. Notice also that the red curve is smooth at 0 while the dashed one has a corner.

The error surface: a bowl

Fix the data (Meena aunty's three flats). Now imagine trying *every* possible pair of weights (w_0, w_1) and computing the loss J for each. Plot J as a height above the (w_0, w_1) plane. You get the **error surface**. This picture makes the whole of Chapter 4 obvious.

Your slide says it in one line: **the error surface is convex with a single global minimum**. Convex means "bowl-shaped, no bumps". Global minimum means "the one lowest point in the whole surface". So there is exactly one best line, and any downhill walk will find it.

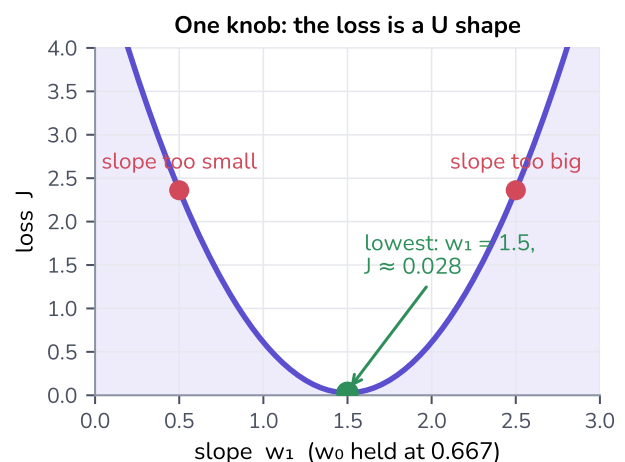
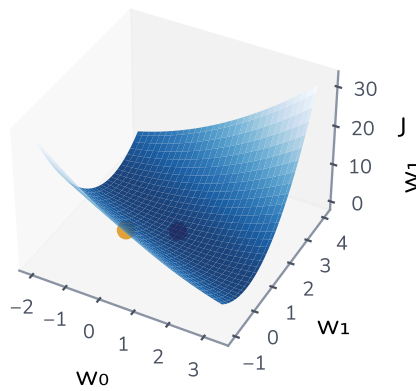


Figure 3.3 One weight first: w_0 fixed at its best value, only the slope w_1 varies. The loss is a U. Slope too small or too big, big loss; at $w_1 = 1.5$, lowest. Your slide 19 shows the same U for its own data.

The error surface is a bowl



Same bowl seen from above (contours)

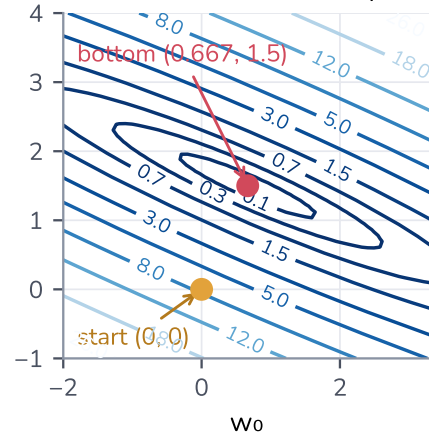


Figure 3.4 Both weights. Left: the loss over the whole (w_0, w_1) plane is a bowl. Right: the same bowl from above. Each curve joins points with the same loss, like height lines on a hiking map: **contour lines**. The bottom (red) is at $(0.667, 1.5)$. Notice the bowl is a long valley: along it the loss changes slowly, across it fast. Chapter 4 starts at the orange dot $(0, 0)$ and walks down.

ON YOUR SLIDES

Slide 17: equation (5), $\ell = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$; equation (6), $J(\mathbf{w}) = \frac{1}{N} \sum_i \frac{1}{2}(\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)})^2$ (Mean Squared Error); equation (7), $\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w})$. Slide 18: the four advantages and equation (8), $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2N}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$. Slide 19: the error-surface pictures, "convex with a single global minimum".

CHECK YOURSELF

1. Why square the error instead of just taking it? **Squaring removes the sign, punishes big errors more, and is smooth so it has a gradient everywhere.**
2. What is $\arg \min_{\mathbf{w}} J(\mathbf{w})$? **The weight vector at which the loss is smallest.**
3. What shape is $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$: a column, a matrix, or a number? **A single number.**
4. What does "convex with a single global minimum" mean for learning? **There is one best answer, and walking downhill from anywhere reaches it.**

IN ONE BREATH

- One example: $\ell = \frac{1}{2}(\hat{y} - y)^2$. All examples: $J(\mathbf{w}) = \frac{1}{N} \sum \frac{1}{2}(\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)})^2$.
- Matrix form: $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$. One number.
- Squared error: smooth, convex, hard on big mistakes, and the "most likely line" under bell-curve noise.
- Over all weights, J is a bowl with one bottom. Find the bottom: that is learning.

Walking downhill: gradient descent

Stand on the bowl, feel which way is steepest, take a small step the other way. Repeat until the ground is flat. That is the whole learning algorithm.

On your slides: **Module 3, slides 20–24** · Gradient descent idea · Batch gradient descent algorithm · Computing the gradient · Update rule · Visualising gradient descent

STORY

Asha draws the bowl on the whiteboard and puts a dot at $(0, 0)$: Nova's starting weights. "Nova cannot see the bowl. It only knows the loss where it stands. How does it get to the bottom?" Ravi thinks of a dark roof at night. "Feel the slope with your feet. Step downhill. Feel again." Asha writes: *gradient* = the slope you feel. *Descent* = step down it.

BIG IDEA

The **gradient** $\nabla J(\mathbf{w})$ is the direction in which the loss rises fastest, and how fast; its opposite is the fastest way down. **Gradient descent** repeats one line: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$. Compute the slope, step against it by a small amount η , repeat. For our bowl, $\nabla J = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$.

THINK OF IT LIKE...

Walking down a hill in fog. You cannot see the valley, but you can feel the ground under your feet. The *gradient* is the direction the ground rises most steeply where you stand; you step the opposite way. The *learning rate* is the length of your step. Tiny steps: you arrive next week. Giant leaps: you jump over the valley. At the bottom the ground is flat, the slope is zero: stop.

The idea

Your slide 20 says: *iteratively update weights in the direction of steepest descent*. Iteratively means "again and again". Steepest descent means "the fastest way down". Here it is with one weight, so the bowl is a U.

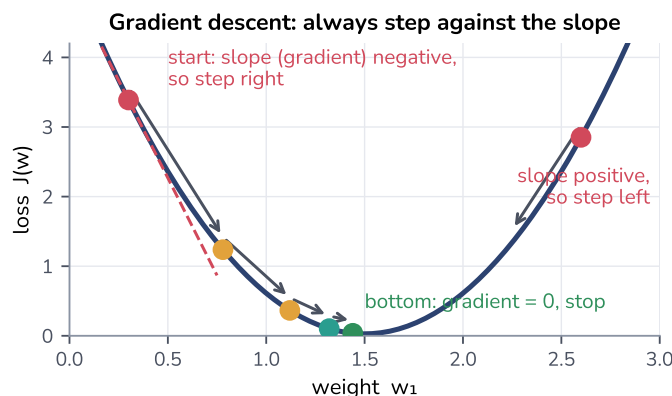


Figure 4.1 One weight, one U-shaped loss. At the left red dot the slope (dashed) is negative, so we increase w_1 ; at the right red dot it is positive, so we decrease w_1 . Either way we move *against* the slope. Notice the steps shrink near the bottom, where the slope is small.

The algorithm

Your slide 21 writes it as "Algorithm 1: Gradient Descent for Linear Regression". Here it is line by line. It is called **batch** gradient descent because every step uses *all* N examples at once (the whole batch); Book 4 will use smaller batches.

1. **Input:** the dataset $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, a learning rate η , a tolerance ε (epsilon, a small number like 10^{-4}).
2. **Initialise** $\mathbf{w}^{(0)} = \mathbf{0}$ (all weights zero) or random. Set the step counter $t = 0$.
3. **Repeat** while not converged: compute the gradient $\nabla J(\mathbf{w}^{(t)}) = \frac{1}{N} \mathbf{X}^\top (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$; update $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$; if $\|\nabla J(\mathbf{w}^{(t+1)})\| < \varepsilon$, stop (converged); otherwise $t \leftarrow t + 1$.
4. **Return** $\mathbf{w}^{(t+1)}$.

The superscript (t) counts steps: $\mathbf{w}^{(0)}$ is the start, $\mathbf{w}^{(1)}$ after one step, and so on. $\|\nabla J\|$ is the length of the gradient vector. When it is nearly zero the ground is nearly flat, so we are at the bottom.

Computing the gradient

Now the one formula this chapter is about. The gradient is a list of slopes, one per weight: "if I nudge w_j up a little, how much does J change?" For squared error there is a clean answer, and the $\frac{1}{2}$ in the loss is what makes it clean.

Σ MATHS, STEP BY STEP

PLAIN MATHS

Start from one example's loss, $\ell = \frac{1}{2}(\mathbf{w}^\top \mathbf{x} - y)^2$. Take its slope with respect to w_j using the chain rule. Bring the power 2 down; it cancels the $\frac{1}{2}$. Keep the bracket. Multiply by the slope of the inside, which is x_j :

$$\frac{\partial \ell}{\partial w_j} = (\mathbf{w}^\top \mathbf{x} - y) x_j$$

Do this for every j at once: the gradient of one example is the mistake times the input vector, $(\mathbf{w}^\top \mathbf{x} - y) \mathbf{x}$.

The total loss is the average over examples, so the total gradient is the average of these:

$$\nabla J(\mathbf{w}) = \frac{\partial J}{\partial \mathbf{w}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Read it as: *each example pushes the weights in the direction of its own input, by an amount equal to its mistake. Average the pushes.*

MATRIX FORM

The column of all mistakes is $\mathbf{X} \mathbf{w} - \mathbf{y}$.

"Multiply each mistake by its input and add up over examples" is exactly what $\mathbf{X}^\top$ times that column does:

$$\nabla J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$$

Check the shapes. $\mathbf{X}^\top$ is $(d+1) \times N$. $\mathbf{X} \mathbf{w} - \mathbf{y}$ is $N \times 1$. So the product is $(d+1) \times 1$: one slope per weight, including the bias. Exactly the shape of $\mathbf{w}$. Good.

Your slides write these shapes out: $\mathbf{X}^\top \in \mathbb{R}^{(d+1) \times N}$, $\mathbf{X} \mathbf{w} - \mathbf{y} \in \mathbb{R}^N$, so $\nabla J \in \mathbb{R}^{d+1}$.

Chapter 5 computes one gradient both ways, entry by entry.

The update rule

With the gradient in hand, the step is one line. Your slide 23 writes it all in one formula, equation (11):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{\eta}{N} \mathbf{X}^\top (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$$

The minus sign is the "walk against the slope". η (eta) is the **learning rate**: it controls the step size. You choose it; the model does not learn it. Such numbers are called **hyperparameters**.

η too large: you may overshoot the minimum: step across the valley, land higher on the other side, step back even further. The loss goes up, and can blow up to infinity. **η too small**: every step is a shuffle; you do go downhill, but it takes thousands of steps. Your slides call this *slow convergence*.

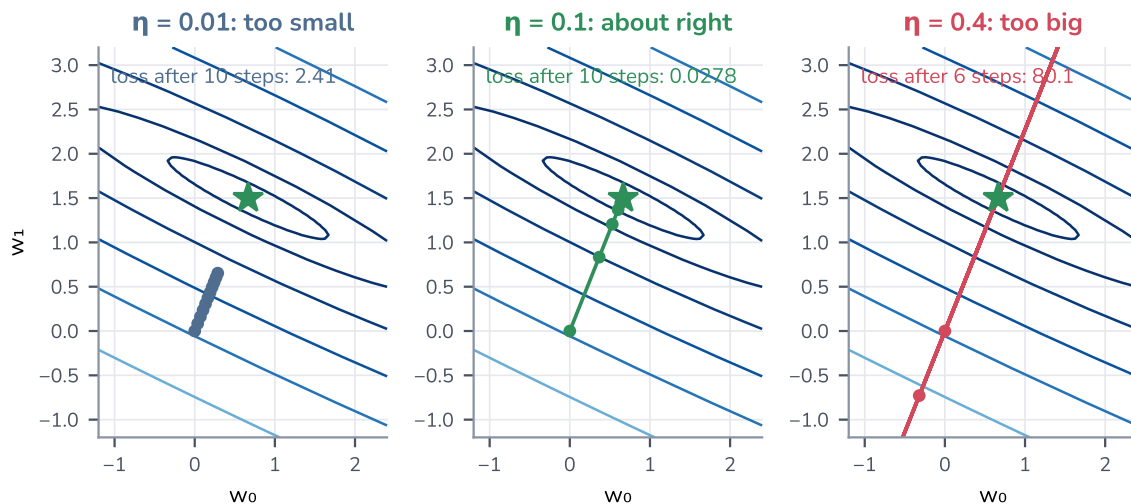


Figure 4.3 Three learning rates, all starting at (0,0), heading for the star. $\eta = 0.01$: ten steps, still far away. $\eta = 0.1$ (your slides' value): ten steps, at the bottom. $\eta = 0.4$: each step jumps over the valley; after six steps the loss is 80, ten times the start, and the path leaves the picture.

The **stopping criterion** is the other choice: stop when $\|\nabla J(\mathbf{w})\| < \epsilon$, or after a maximum number of iterations (Chapter 7 adds two more).

The contour picture

Here is the deck's example run, from (0,0) with $\eta = 0.1$, on the contour map from Chapter 3. Chapter 5 computes every one of these numbers by hand.

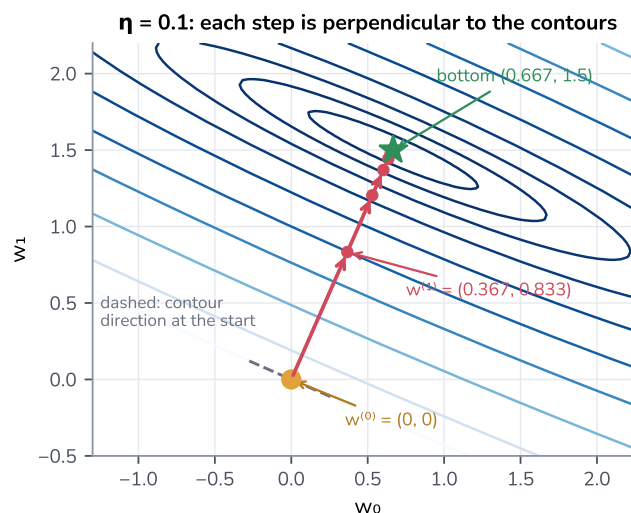


Figure 4.4 Gradient descent from above; each red dot is one $\mathbf{w}^{(t)}$. Your slide: *each step moves perpendicular to contour lines toward the minimum*. The first step leaves (0,0) at a right angle to the contour through it (dashed). Along a contour the loss does not change, so the steepest way down is straight across. Notice the steps shrink as the gradient shrinks.

? STOP AND THINK

Ravi: "The first step points roughly at the bottom, but not exactly. Why not just aim at the bottom?"

✓ OUR ANSWER

Because Nova does not know where the bottom is; that is the whole point. It only knows the slope under its feet. In a long, stretched valley the steepest direction across the contours is not the direction of the bottom, so the path curves. Chapter 7's feature scaling makes the valley rounder and the path straighter.

☰ ON YOUR SLIDES

Slide 20: the idea, a ball on the loss curve, " $\eta > 0$ is the learning rate (step size)". Slide 21: Algorithm 1, lines 1–9. Slide 22: equation (9), $\nabla J(\mathbf{w}) = \frac{1}{N} \sum_i (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$, and equation (10), $\frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$, with shapes. Slide 23: equation (11), the update rule; "too large: may overshoot; too small: slow convergence"; the stopping criterion. Slide 24: the contour picture, "each step moves perpendicular to contour lines".

! CAREFUL

The sign. The update is $\mathbf{w} - \eta \nabla J$, with a *minus*. Write plus and you climb the bowl instead; the loss grows every step. Your slide 41 lists "wrong sign in update rule" as a classic bug. Also: the mistake is $\hat{y} - y$ (guess minus truth), not $y - \hat{y}$; swapping it flips the gradient, the same bug in disguise.

✓ CHECK YOURSELF

1. What does the gradient tell you? **The direction in which the loss rises fastest, and how steeply.**
2. Why is there a minus sign in the update? **We want to go downhill, against the gradient.**
3. What are the shapes in $\mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$? **$(d+1) \times N$ times $N \times 1$ gives $(d+1) \times 1$.**
4. Loss goes up after every step. Most likely cause? **Learning rate too large (or a sign error).**
5. When does the algorithm stop? **When the gradient's length is below ε , or at the maximum number of iterations.**

● IN ONE BREATH

- Gradient = the uphill direction. Step the other way: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$.
- For squared error: $\nabla J = \frac{1}{N} \sum (\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)} = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$, shape $(d+1) \times 1$.
- η too big: overshoot, loss explodes. Too small: slow. Stop when $\|\nabla J\| < \varepsilon$ or after max iterations.
- Each step is perpendicular to the contour lines. Steps shrink near the bottom.

Nova's first price guess, number by number

Three flats, two weights, one learning step, every multiplication written out. Then ten more steps in a table, and the line Nova draws after each.

On your slides: **Module 3, slides 26–31** · Numerical example: setup · Initialisation · Computing gradient · Weight update · New loss · Computational graph

STORY

Meena aunty picks three flats from her notebook. "Small, medium, large," she says. "Sizes 1, 2, 3. They sold for 2, 4, 5." Asha types them into Nova and sets both weights to zero. Nova's tiny screen shows its first guesses: 0, 0, 0. "It thinks every flat is free," Ravi laughs. "Watch," says Asha, and presses the button once.

BIG IDEA

One step of gradient descent is five small computations: guess ($\hat{y} = \mathbf{X}\mathbf{w}$), loss (J), mistakes ($\mathbf{e} = \hat{y} - \mathbf{y}$), gradient ($\frac{1}{N}\mathbf{X}^\top \mathbf{e}$), update ($\mathbf{w} - \eta \nabla J$). On the deck's three flats, the very first step cuts the loss from 7.5 to 1.51, an 80% drop.

THINK OF IT LIKE...

Tuning a radio by hand. You start anywhere (both knobs at zero). You listen to the noise (the loss). You feel which way each knob should turn and by how much (the gradient). You turn both knobs a little (the update). The first turn removes most of the hiss. The next turns are smaller and smaller, until the station is clear.

Setup

Meena aunty's three flats

Size x_1	Price y
1	2
2	4
3	5

$N = 3$ examples, $d = 1$ feature

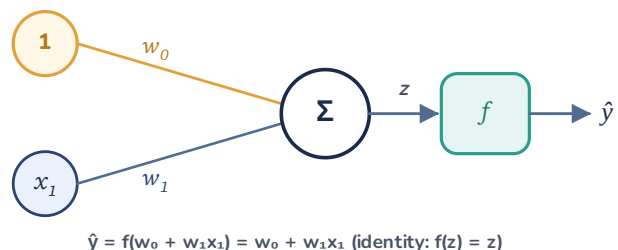


Figure 5.1 The tiny dataset from your slide 26 and the neuron that will learn it. One feature (size), so one weight w_1 plus the bias w_0 . Everything below is these two numbers being nudged.

In matrix form, with the ones column in front, and the hyperparameters from your slide 27:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \in \mathbb{R}^3, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \in \mathbb{R}^2, \quad \eta = 0.1, \quad N = 3$$

WORKED EXAMPLE, PART 1: GUESS, LOSS, GRADIENT (SLIDES 27–28)

Initialise. $\mathbf{w}^{(0)} = (0, 0)$.

- 1 **Predict:** $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)}$. Row 1: $1 \cdot 0 + 1 \cdot 0 = 0$. Row 2: $1 \cdot 0 + 2 \cdot 0 = 0$. Row 3: $1 \cdot 0 + 3 \cdot 0 = 0$.
So $\hat{\mathbf{y}}^{(0)} = (0, 0, 0)$.
zero weights give zero guesses
- 2 **Loss:** mistakes are $0 - 2, 0 - 4, 0 - 5$. Squares: 4, 16, 25. $J(\mathbf{w}^{(0)}) = \frac{1}{2N}(4 + 16 + 25) = \frac{1}{6} \times 45 = 7.5$.
 $J = (1/2N) \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$
- 3 **Error vector:** $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = (0 - 2, 0 - 4, 0 - 5) = (-2, -4, -5)$.
guess minus truth
- 4 **Gradient, first entry** (bias): $\frac{1}{3}[1 \cdot (-2) + 1 \cdot (-4) + 1 \cdot (-5)] = \frac{1}{3}(-11) = -3.67$.
first row of $\mathbf{X}^T$ is (1, 1, 1)
- 5 **Gradient, second entry** (size): $\frac{1}{3}[1 \cdot (-2) + 2 \cdot (-4) + 3 \cdot (-5)] = \frac{1}{3}(-2 - 8 - 15) = \frac{1}{3}(-25) = -8.33$.
second row of $\mathbf{X}^T$ is (1, 2, 3)
- 6 So $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{N}\mathbf{X}^T\mathbf{e}^{(0)} = (-3.67, -8.33)$.
both negative: both weights should go up

WORKED EXAMPLE, PART 2: UPDATE, NEW GUESSES, NEW LOSS (SLIDES 29–30)

- 7 **Update:** $w_0^{(1)} = 0 - 0.1 \times (-3.67) = 0 + 0.367 = 0.367$.
 $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$; minus a negative is plus
 - 8 $w_1^{(1)} = 0 - 0.1 \times (-8.33) = 0 + 0.833 = 0.833$. So $\mathbf{w}^{(1)} = (0.367, 0.833)$.
 - 9 **New predictions:** $\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)}$. Row 1: $0.367 \cdot 1 + 0.833 \cdot 1 = 1.20$. Row 2: $0.367 + 0.833 \cdot 2 = 0.367 + 1.667 = 2.03$. Row 3: $0.367 + 0.833 \cdot 3 = 0.367 + 2.50 = 2.87$.
one row of $\mathbf{X}$ at a time
 - 10 **New error vector:** $\mathbf{e}^{(1)} = (1.20 - 2, 2.03 - 4, 2.87 - 5) = (-0.80, -1.97, -2.13)$.
 - 11 **New loss:** squares 0.64, 3.88, 4.54. $J(\mathbf{w}^{(1)}) = \frac{1}{6}(0.64 + 3.88 + 4.54) = \frac{1}{6} \times 9.06 \approx 1.51$.
 $(1/2N) \|\mathbf{e}^{(1)}\|^2$
 - 12 **Progress:** $7.5 \rightarrow 1.51$. Drop = $(7.5 - 1.51)/7.5 = 0.80$, an **80% reduction** in one step.
- Continue until $\|\nabla J(\mathbf{w})\| < \varepsilon$ or the maximum number of iterations is reached.

! CAREFUL

Rounding. The slides round to two or three decimals at each step ($-3.67, 0.367, 1.20$). With exact fractions ($-11/3, 11/30, 6/5$) you get $J(\mathbf{w}^{(1)}) = 1.5098$, which also rounds to 1.51. Last-digit differences are rounding, not mistakes. In an exam, round the way the slides do.

The same step as a picture

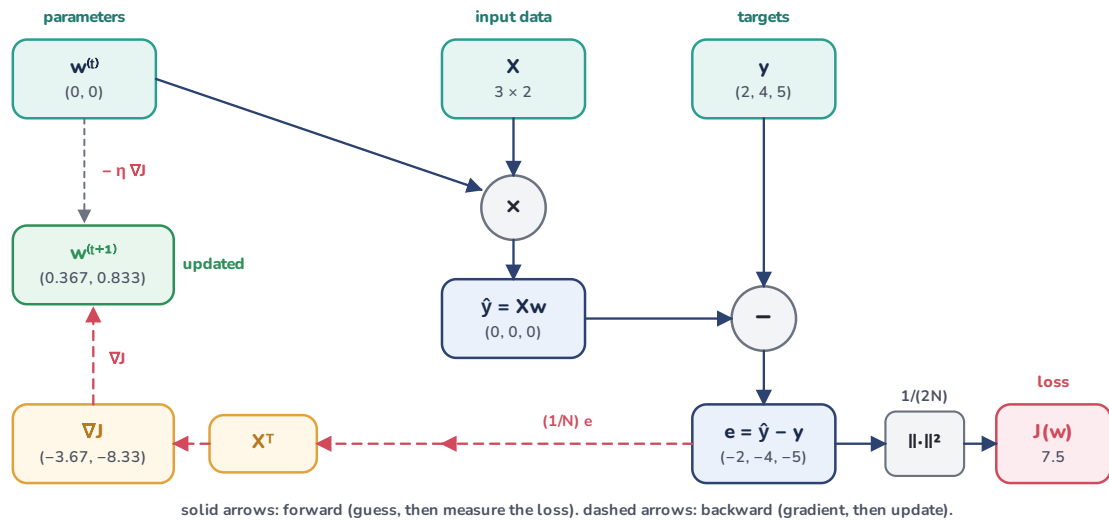


Figure 5.2 Your slide 31 draws the step as a **computational graph**: boxes for numbers, circles for operations, arrows for "flows into", here with the first iteration's numbers. Solid arrows: $\mathbf{w}^{(t)}$ and $\mathbf{X}$ meet at $\times$ to give $\hat{\mathbf{y}}$; $\hat{\mathbf{y}}$ and $\mathbf{y}$ meet at $-$ to give $\mathbf{e}$; squaring and scaling $\mathbf{e}$ gives J . Dashed arrows, backwards: $\mathbf{e}$ times $\frac{1}{N}$, times $\mathbf{X}^T$, is ∇J ; times $-\eta$, it is the change to $\mathbf{w}$. Notice the backward path reuses $\mathbf{e}$. Book 5 stretches this picture across many layers and calls it backpropagation.

Ten more steps

Nova keeps pressing the button. Here are the weights, loss and gradient after each step, computed with the same five lines. Rows 0 and 1 are the numbers you just worked out.

t	w_0	w_1	$J(\mathbf{w}^{(t)})$	$\partial J / \partial w_0$	$\partial J / \partial w_1$	$\ \nabla J\ $
0	0.000	0.000	7.500	-3.667	-8.333	9.104
1	0.367	0.833	1.510	-1.633	-3.711	4.055
2	0.530	1.204	0.322	-0.728	-1.653	1.806
3	0.603	1.370	0.086	-0.324	-0.736	0.804
4	0.635	1.443	0.039	-0.145	-0.328	0.358
5	0.650	1.476	0.030	-0.065	-0.146	0.160
6	0.656	1.491	0.028	-0.029	-0.065	0.071
7	0.659	1.497	0.028	-0.013	-0.029	0.032
8	0.660	1.500	0.028	-0.006	-0.013	0.014
9	0.661	1.501	0.028	-0.003	-0.005	0.006
10	0.661	1.502	0.028	-0.002	-0.002	0.003

Notice three things. The loss never goes up: every step is downhill. The gradient shrinks by more than half each step, so the steps shrink too. And the loss stops at 0.028, not 0: the three flats are not on one line, so no line gets every price right. The best line is $\mathbf{w}^* = (0.667, 1.5)$; after ten steps Nova is within a few thousandths of it.

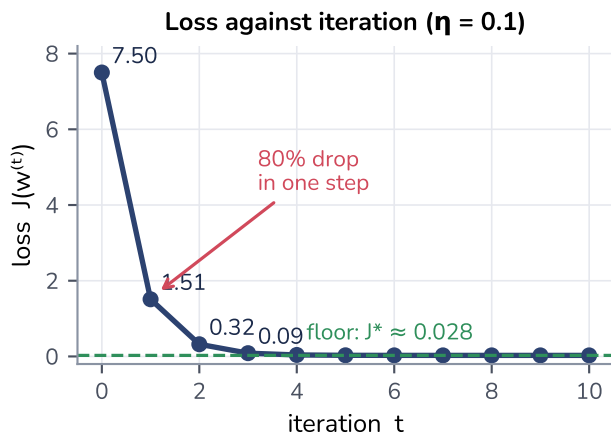


Figure 5.3 The loss column of the table as a curve: a steep fall, then a long flat tail. Almost all the learning happens in the first three steps. Dashed: the lowest loss any line can reach.

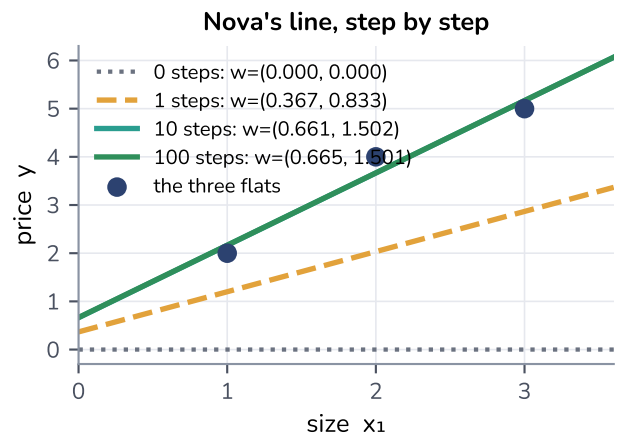


Figure 5.4 Nova's line after 0, 1, 10 and 100 steps: flat at zero; lifted but still too low; among the dots; and barely moved, since it was already at the bottom.

? STOP AND THINK

Ravi: "The loss went 1.51, then 0.028. Will it reach 0 if we wait?"

✓ OUR ANSWER

No. The dots (1, 2) and (2, 4) lie on $y = 2x$; the third would need $y = 6$, but it is 5. No line passes through all three. The best line balances them, leaving mistakes $+0.17, -0.33, +0.17$ and loss 0.028.

📖 ON YOUR SLIDES

Slide 26: the table, the neuron, $\mathbf{X} \in \mathbb{R}^{3 \times 2}$, $\mathbf{y} \in \mathbb{R}^3$, $\mathbf{w} \in \mathbb{R}^2$. Slide 27: $\eta = 0.1$, $N = 3$, $\mathbf{w}^{(0)} = \mathbf{0}$, $\hat{\mathbf{y}}^{(0)} = \mathbf{0}$, $J(\mathbf{w}^{(0)}) = \frac{1}{6}(4 + 16 + 25) = 7.5$. Slide 28: $\mathbf{e}^{(0)} = (-2, -4, -5)$, $\nabla J = \frac{1}{N}\mathbf{X}^\top \mathbf{e}^{(0)} = (-3.67, -8.33)$. Slide 29: $\mathbf{w}^{(1)} = (0.367, 0.833)$, $\hat{\mathbf{y}}^{(1)} = (1.20, 2.03, 2.87)$. Slide 30: $\mathbf{e}^{(1)} = (-0.80, -1.97, -2.13)$, $J(\mathbf{w}^{(1)}) \approx 1.51$, "80% reduction!". Slide 31: the computational graph.

✓ CHECK YOURSELF

- Why is the initial loss 7.5 and not 45? **Because J divides the sum of squares (45) by $2N = 6$.**
- The gradient at $\mathbf{w}^{(0)}$ is $(-3.67, -8.33)$. Which way do the weights move? **Up, both of them: minus a negative gradient is a positive change.**
- What is $\mathbf{w}^{(1)}$? **(0.367, 0.833).**
- Why does the loss settle at 0.028 rather than 0? **The three points are not on one straight line, so no line has zero error.**

🗨 IN ONE BREATH

- Start: $\mathbf{w}^{(0)} = (0, 0)$, guesses (0, 0, 0), loss 7.5, error $(-2, -4, -5)$.
- Gradient $\frac{1}{3}\mathbf{X}^\top \mathbf{e} = (-3.67, -8.33)$. Update with $\eta = 0.1$: $\mathbf{w}^{(1)} = (0.367, 0.833)$.
- New guesses (1.20, 2.03, 2.87), error $(-0.80, -1.97, -2.13)$, loss 1.51: an 80% drop.
- Ten steps later the loss is 0.028, the floor for these flats, with $\mathbf{w} \approx (0.661, 1.502)$.

Was it a good guess? Evaluation

A model is judged on flats it has never seen. Four numbers tell you how good it is: MSE, RMSE, MAE and R^2 .

On your slides: **Module 3, slides 33–35** · Training vs test error · Evaluation metrics: MSE, RMSE, MAE · R^2 score

STORY

"Nova has learnt my three flats," says Meena aunty. "Loss 0.028. Very good." Asha shakes her head. "Nova has learnt those three. I need to know if it can price a flat it has never seen." Meena aunty flips to a fresh page in her notebook: four more flats, sold last month, that Nova never saw. "Then test it on these."

BIG IDEA

Keep some data aside and never learn from it. That is the **test set**. The error on the test set, not the training set, tells you whether the model will work in the real world. To report that error, use **MSE** (average squared mistake), **RMSE** (its square root, in the units of the answer), **MAE** (average absolute mistake) and **R^2** (how much better than always guessing the average).

THINK OF IT LIKE...

Practice questions and the real exam. If the exam were the same questions you practised, a perfect score would prove nothing; you could have memorised the answers. So the exam uses questions you have not seen. Training data is the practice set. Test data is the exam. A model that scores well on practice and badly on the exam has memorised instead of learnt.

Training error and test error

Your slide 33 gives the key principle: **evaluate on unseen data**. Split the data before you start.

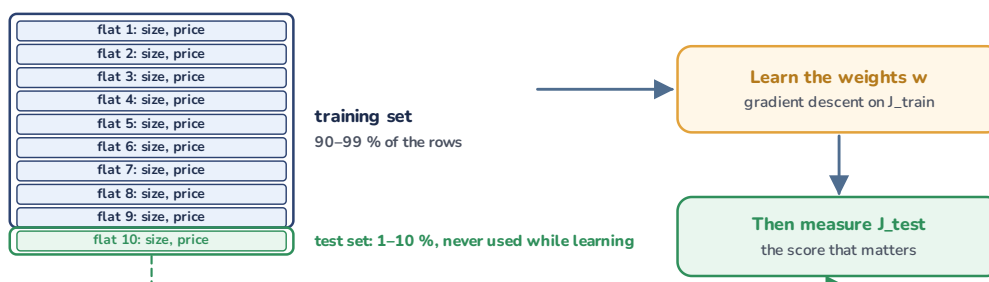


Figure 6.1 The split. Most rows (typically 90–99%) form the **training set**, used to learn the weights $\mathbf{w}$. A few rows (1–10%) form the **test set**, locked away until the end. Notice the dashed arrow: the test rows only ever meet the finished model.

PLAIN MATHS

The same squared mistake, averaged over two different sets of rows:

$$J_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} (\hat{y}^{(i)} - y^{(i)})^2$$

$$J_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i \in \text{test}} (\hat{y}^{(i)} - y^{(i)})^2$$

N_{train} and N_{test} are the numbers of rows in each set. Goal: make J_{test} small. Your slides call this **generalisation performance**: doing well on data you did not train on.

→

MATRIX FORM

Split the design matrix and the targets by rows, then apply the same formula to each half:

$$J_{\text{test}} = \frac{1}{N_{\text{test}}} \|\mathbf{X}_{\text{test}} \mathbf{w} - \mathbf{y}_{\text{test}}\|^2$$

The weights $\mathbf{w}$ were learnt from $\mathbf{X}_{\text{train}}$ only. Notice there is no $\frac{1}{2}$ here: these are the plain averages used for reporting, not the loss used for learning.

Four ways to report the error

All four start from the same list of mistakes $\hat{y}^{(i)} - y^{(i)}$ on the test set. They differ in how they squash the list into one number.

DEFINITION

1. Mean Squared Error (MSE). Square each mistake, average:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$$

2. Root Mean Squared Error (RMSE). The square root of MSE. Squaring changed the units (price units became price units squared); the root changes them back, so RMSE reads like a typical mistake:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2}$$

3. Mean Absolute Error (MAE). Drop the sign of each mistake, average. Your slides note it is *less sensitive to outliers* than MSE: one huge mistake does not dominate.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - \hat{y}^{(i)}|$$

4. R^2 score (coefficient of determination). Compare the model's squared mistakes to those of the dumbest possible model, "always guess the average price $\bar{y}$ ":

$$R^2 = 1 - \frac{\sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^N (y^{(i)} - \bar{y})^2} = 1 - \frac{\text{SS}_{\text{residual}}}{\text{SS}_{\text{total}}}, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y^{(i)}$$

SS stands for "sum of squares". $\text{SS}_{\text{residual}}$ is the model's leftover error; SS_{total} is how much the prices vary on their own. Your slides say R^2 is the *proportion of variance explained by the model*.

WORKED EXAMPLE: FOUR UNSEEN FLATS

Meena aunty's four test flats sold for $y = (3, 4, 6, 7)$. Nova's guesses are $\hat{y} = (3.5, 3.5, 6.5, 8.0)$. $N = 4$.

- 1 Mistakes $\hat{y} - y$: $3.5 - 3 = 0.5$; $3.5 - 4 = -0.5$; $6.5 - 6 = 0.5$; $8.0 - 7 = 1.0$.
- 2 Squares: 0.25, 0.25, 0.25, 1.0. Sum = 1.75.
- 3 $MSE = 1.75/4 = 0.4375$.
average of the squares
- 4 $RMSE = \sqrt{0.4375} = 0.661$ price units.
back to the units of a price: a typical miss of about 0.66
- 5 Absolute mistakes: 0.5, 0.5, 0.5, 1.0. Sum = 2.5. $MAE = 2.5/4 = 0.625$.
smaller than RMSE: the big mistake (1.0) counts less without squaring
- 6 Mean price $\bar{y} = (3 + 4 + 6 + 7)/4 = 20/4 = 5$.
- 7 Deviations from the mean $y - \bar{y}$: $-2, -1, 1, 2$. Squares 4, 1, 1, 4. $SS_{total} = 10$.
how badly "always guess 5" would do
- 8 $SS_{residual} = 1.75$ (from step 2). $R^2 = 1 - 1.75/10 = 1 - 0.175 = 0.825$.
the model removes 82.5% of the variation; in the "good fit" range

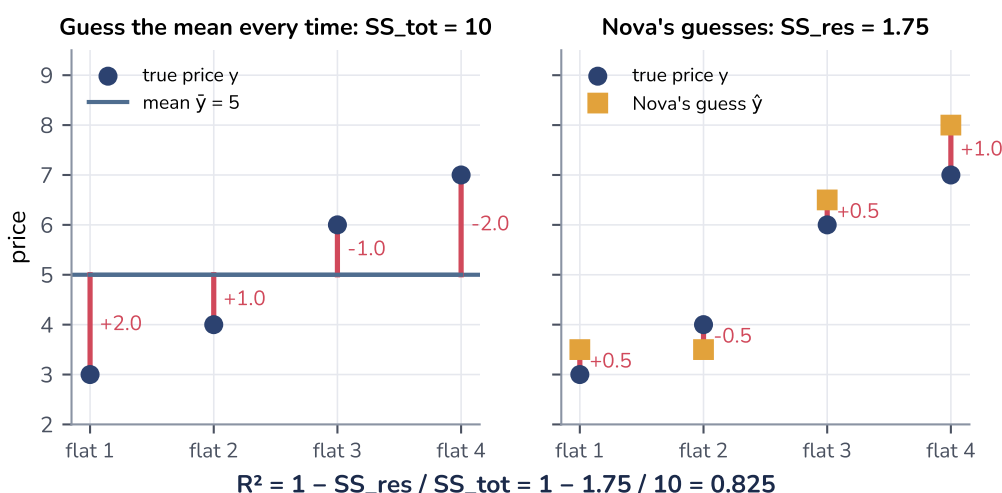


Figure 6.2 R^2 as a picture. Left: the red sticks are the gaps between each true price and the mean, 5. Their squares add to $SS_{total} = 10$. Right: the gaps between each true price and Nova's guess. Their squares add to $SS_{residual} = 1.75$. Notice how much shorter the right-hand sticks are: that shrinkage, $1 - 1.75/10$, is R^2 .

HOW TO READ R^2 (YOUR SLIDE 35)

Value	Meaning	On the four flats
$R^2 = 1$	Perfect fit: all variance explained.	guesses (3, 4, 6, 7): $SS_{res} = 0$, $R^2 = 1$
$R^2 = 0$	No better than predicting the mean.	guesses (5, 5, 5, 5): $SS_{res} = 10$, $R^2 = 0$
$R^2 < 0$	Worse than the mean. Possible on a test set.	guesses (7, 7, 7, 7): $SS_{res} = 16 + 9 + 1 + 0 = 26$, $R^2 = -1.6$
$0.7 < R^2 < 0.9$	Typical of a good fit.	Nova's 0.825 sits here

? STOP AND THINK

Suppose Nova's fourth guess had been 10 instead of 8: a miss of 3 on one flat. What happens to MSE and MAE?

✓ OUR ANSWER

Squares become 0.25, 0.25, 0.25, 9, sum 9.75, so MSE jumps from 0.4375 to $9.75/4 = 2.4375$, more than five times bigger. MAE only goes from 0.625 to $(0.5 + 0.5 + 0.5 + 3)/4 = 1.125$, less than double. One outlier dominates MSE but not MAE. That is what "less sensitive to outliers" means. (R^2 falls to $1 - 9.75/10 = 0.025$: barely better than guessing the mean.)

☰ ON YOUR SLIDES

Slide 33: "evaluate on unseen data", train 90–99%, test 1–10%, J_{train} and J_{test} , "minimise test error (generalisation performance)". Slide 34: equations (12) MSE, (13) $\text{RMSE} = \sqrt{\text{MSE}}$, (14) MAE, "less sensitive to outliers than MSE". Slide 35: equation (15), $R^2 = 1 - \text{SS}_{\text{residual}}/\text{SS}_{\text{total}}$, $\bar{y}$ the mean, and the four readings (1, 0, negative, typical 0.7–0.9).

! CAREFUL

Never tune anything on the test set. If you try ten learning rates and keep the one with the lowest test error, the test set has quietly become training data. Its score is no longer honest. Keep a third slice, a *validation set*, for such choices (Chapter 7 uses one for early stopping). The test set is opened once, at the end.

✓ CHECK YOURSELF

1. Why not judge a model by its training error? **It may have memorised the training rows; only unseen rows show whether it generalises.**
2. MSE is 0.4375. What is RMSE, and why is it more readable? **$\sqrt{0.4375} = 0.661$; it is in price units, like a typical mistake.**
3. Which is hurt more by one huge mistake, MSE or MAE? **MSE, because it squares the mistake.**
4. What does $R^2 = 0$ mean? **The model does no better than always predicting the average.**
5. Can R^2 be negative? **Yes, on a test set, when the model is worse than the mean.**

🔴 IN ONE BREATH

- Split the data. Learn on the training set. Judge on the test set. Aim for low test error.
- MSE = average squared mistake. RMSE = its root, in answer units. MAE = average absolute mistake, kinder to outliers.
- $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$: 1 is perfect, 0 is "same as the mean", negative is worse, 0.7–0.9 is good.
- Four flats: MSE 0.4375, RMSE 0.661, MAE 0.625, $R^2 = 0.825$.

Tips, debugging, and the big picture

How to pick the learning rate, when to stop, why to scale your features, what to check when the loss misbehaves, and the one picture that holds the whole module.

On your slides: **Module 3, slides 37–45** · Python demo · Tips for implementation · More practical tips · Debugging checklist · Summary · Key takeaways · Linear regression in one slide

STORY

Meena aunty wants more: not just size, but square feet, floor number and distance to the metro in metres. Asha adds the columns and presses the button. The loss reads 7.5, then 340, then 15,000, then *NaN*. "It broke," says Ravi. "No. I made three mistakes at once," says Asha, "and there is a checklist for all three."

BIG IDEA

Gradient descent is simple, but a few choices decide whether it works: the learning rate, when to stop, how to start, and which features to use. Above all: are the features on similar scales? Each way of failing shows up in the loss curve, and each has a known fix.

THINK OF IT LIKE...

Cooking on a new stove. Too high a flame and the dish burns (loss explodes). Too low and dinner takes all night (loss barely moves). You taste as you go (plot the loss) and stop when the taste stops improving (early stopping). And you cut the vegetables to one size so they cook at one speed: feature scaling, the tip people forget.

Tip 1: choosing the learning rate

Your slide 37 points to demo code and a virtual lab (Taxila); these tips are what you tune there.

- Start with $\eta \in \{0.001, 0.01, 0.1, 1.0\}$: four values, each ten times the last.
- Plot the loss against iterations for each one.
- If the loss **increases or becomes NaN**, decrease η . (NaN means "not a number": the values overflowed.)
- If convergence is **too slow**, increase η .
- Use a **learning-rate schedule** for fine-tuning: a bigger η early, then smaller as you approach the bottom.

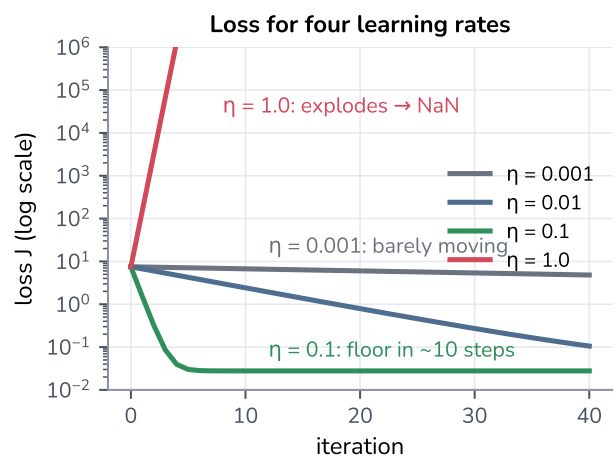


Figure 7.1 Four learning rates on the three flats, log scale. $\eta = 1.0$ explodes to NaN; $\eta = 0.001$ is nearly flat, still at 6.7 after ten steps; $\eta = 0.1$ hits the floor in ten. Pick the steepest curve that does not explode.

Tip 2: when to stop training

- **Gradient is tiny:** $\|\nabla J\| < \varepsilon$, for example $\varepsilon = 10^{-4}$. The ground is flat: you are at the bottom.
- **Loss stopped changing:** $|J^{(t)} - J^{(t-1)}| < \varepsilon$. Another step would gain nothing.
- **Maximum iterations reached:** for example $T = 1000$ or $10,000$. A safety net so the loop always ends.
- **Validation loss starts rising:** keep a validation set aside (Chapter 6). When its loss goes up while the training loss keeps falling, the model has started to memorise the training rows instead of learning the pattern. Stop there and keep those weights. This is **early stopping**.

Tip 3: weight initialisation

Zero initialisation, $\mathbf{w}^{(0)} = \mathbf{0}$, works well for linear regression: the bowl has one bottom, so where you start only changes how long the walk is. Avoid large initial values; they start you high on a steep slope, and the first update can overshoot or overflow. (In Book 5, with hidden layers, zero will no longer be fine.)

Tip 4: feature engineering

- **Domain knowledge:** create meaningful derived features. Meena aunty thinks in "price per square foot"; a feature like that can make the straight-line assumption much more true.
- **Feature selection:** remove irrelevant or redundant features. A column that is pure noise still gets a weight, and that weight is one more thing to get wrong.

Tip 5: feature scaling

Suppose one feature is "size in square feet" (500 to 1500) and another is "floor" (1 to 10). A change of 1 in the weight for square feet moves every prediction by hundreds; a change of 1 in the floor weight moves it by a few units. The bowl becomes a long, narrow valley. Gradient descent then zig-zags across the valley and crawls along it: *features with different scales slow convergence*. The fix is to put every feature on the same ruler before training. Your slides give two rulers.

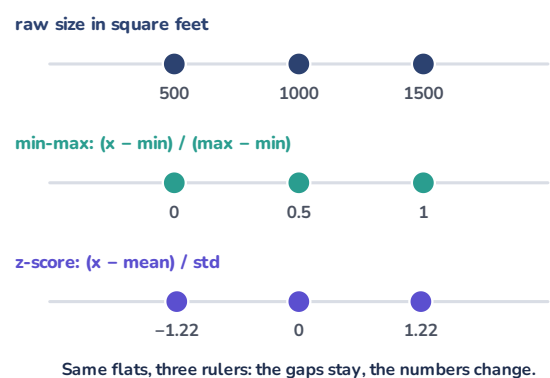


Figure 7.3 Three flats, three rulers. Notice that the spacing never changes; only the numbers do. Min-max puts them in $[0, 1]$; the z-score centres them on 0.

MIN-MAX SCALING

Squeeze feature j into the range $[0, 1]$:

$$x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$$

The smallest value becomes 0, the largest becomes 1, everything else lands in between.

For square feet 500, 1000, 1500: $(500 - 500)/1000 = 0$, $(1000 - 500)/1000 = 0.5$, $(1500 - 500)/1000 = 1$.

STANDARDISATION (Z-SCORE)

Give feature j mean 0 and standard deviation 1:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

μ_j (mu) is the mean of the column and σ_j (sigma) is its standard deviation, the typical distance from the mean. For 500, 1000, 1500: mean 1000, standard deviation 408.2, so $(500 - 1000)/408.2 = -1.22$, 0, $+1.22$.

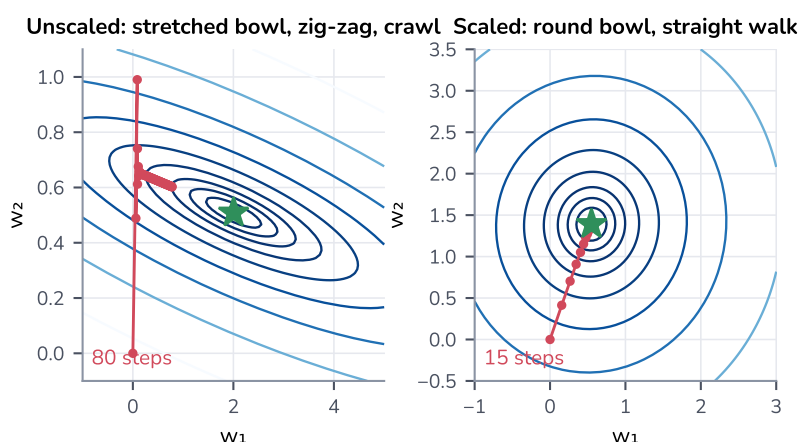


Figure 7.4 Why it matters. Left: features on very different scales give a stretched bowl; gradient descent zig-zags, then crawls, and after 80 steps is still not there. Right: the same data z-scored gives a round bowl; 15 steps walk straight to the bottom.

! CAREFUL

Compute min, max, μ and σ on the *training* set only, then apply the same numbers to the test set and to every new flat. If you rescale the test set with its own statistics, the test rows are on a different ruler from the one the weights were learnt on. Also: do not scale the ones column.

Debugging checklist

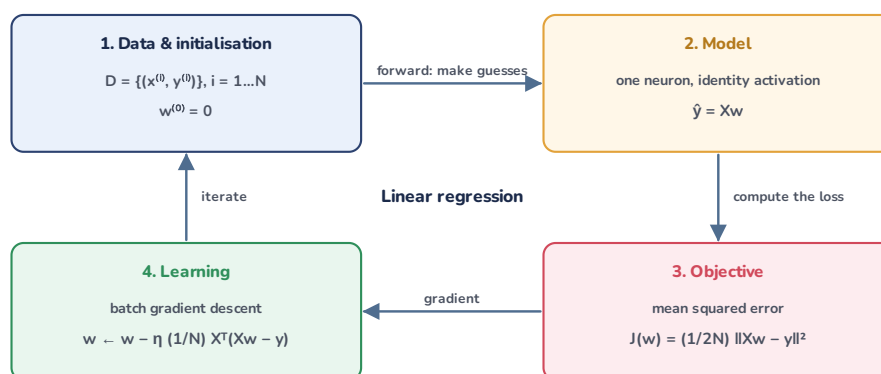
Your slide 41, as a table. Match the symptom in the left column to what Nova's screen is showing.

Symptom	Likely causes	Fix
Loss is NaN or Inf	Learning rate too large. Numerical overflow: check the feature scales.	Decrease η . Scale the features.
Loss oscillates (up, down, up, down)	Learning rate too large: each step jumps across the valley.	Decrease η , or use learning-rate decay (shrink η over time).
Loss not decreasing	Learning rate too small. A bug in the gradient (check the signs!). Wrong sign in the update rule (should be minus).	Increase η . Verify the code against Chapter 5's numbers.
Training works, test error high	Overfitting: too many features relative to the amount of data.	Regularisation (Book 5), more data, or a simpler model.

Asha's three mistakes from the story: square feet unscaled (overflow), a learning rate tuned for the old features (too large now), and a floor feature nobody needed. Scale, lower η , drop the column.

Putting it all together

Your slide 43 draws the whole module as four boxes in a loop. Here it is, with everything from Chapters 2 to 5 written into the boxes.



The same four boxes come back for every model in this course — only the contents of boxes 2 and 3 change.

Figure 7.5 Linear regression as the four-component loop. Data and a starting $\mathbf{w}$ go into the model (one neuron, identity activation), which makes guesses. The objective (MSE) scores them. The learning algorithm (batch gradient descent) uses the gradient to move $\mathbf{w}$. Then round again. Your slide adds: *this framework extends to more complex models: neural networks, deep learning!*

KEY TAKEAWAYS (YOUR SLIDE 44)

- Regression predicts continuous outputs from input features.
- Linear regression assumes a linear relationship: $y = \mathbf{w}^\top \mathbf{x} + w_0$.
- A single neuron with identity activation implements linear regression.
- Squared error loss gives a convex objective: one bowl, one bottom.
- Gradient descent optimises the weights step by step: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$.
- Feature scaling is critical for fast convergence.
- Evaluation on a test set ensures generalisation.

LINEAR REGRESSION ON ONE SLIDE (YOUR SLIDE 45)

- **Problem:** predict continuous values, $y \in \mathbb{R}$.
- **Model:** a single neuron with identity activation.

- **Output:** $\hat{y} = \mathbf{w}^\top \mathbf{x} \in (-\infty, \infty)$.
- **Loss:** mean squared error.
- **Training:** batch gradient descent.

Key insight: a linear relationship between inputs and output, optimised by minimising squared errors. The slide's picture is Figure 2.2 with two inputs, $f(z) = z$, and a continuous output.

📄 ON YOUR SLIDES

Slide 37: demo Python code, virtual lab (Labware on Taxila). Slide 39: tip 1 (learning rate) and tip 2 (when to stop). Slide 40: tips 3 (initialisation), 4 (feature engineering) and 5 (scaling: min-max $x'_j = (x_j - \min)/(\max - \min)$, z-score $x'_j = (x_j - \mu_j)/\sigma_j$). Slide 41: debugging checklist. Slide 43: the four-box summary. Slide 44: key takeaways. Slide 45: linear regression in one slide.

✅ CHECK YOURSELF

1. The loss prints NaN after three steps. First two things to try? **Decrease the learning rate; scale the features.**
2. What is early stopping? **Stop training when the validation loss starts rising, even if the training loss is still falling.**
3. Min-max scale the values 2, 4, 10. **0, 0.25, 1.**
4. Why is zero initialisation fine here? **The loss is a bowl with one bottom, so any start reaches it.**
5. Training error tiny, test error huge: name and fix? **Overfitting. More data, fewer features, or regularisation.**

🕒 IN ONE BREATH

- Learning rate: try 0.001, 0.01, 0.1, 1.0, plot, pick the fastest curve that does not explode.
- Stop when the gradient or the loss change is tiny, at max iterations, or when validation loss rises.
- Start at zero. Engineer and select features. **Scale** them (min-max or z-score) so the bowl is round.
- NaN → smaller η and scaling; oscillating → smaller η ; flat → bigger η or a sign bug; good train but bad test → overfitting.

What Nova still cannot do

Nova can now answer "how much": the price of a flat, and how loud to reply. But the question Asha started with is still open. When someone speaks, Nova must decide: *was that my name, or not?* That is not "how much". It is "which one". A line that outputs any number is the wrong tool for a yes-or-no answer. Book 4 gives the neuron a new activation, a new loss, and the power to choose.

Where you are in the series

Six small books, one story. Asha builds Nova, a robot that must learn to hear its name — and along the way she meets every idea in the Deep Neural Network course, in the order the course teaches them.

BOOK 1

Teaching a Machine to Listen

What deep learning is, and the four parts of every learning problem

BOOK 2

One Tiny Brain Cell

The artificial neuron, the perceptron, logic gates and the XOR wall

BOOK 3

Guessing How Much

A linear neuron for regression, squared loss and gradient descent

BOOK 4

Guessing Which One

Sigmoid and softmax neurons, cross-entropy, SGD and mini-batches

BOOK 5

A Real Brain

Hidden layers, activations, backpropagation by hand, and design

BOOK 6

Eyes for the Machine

Convolution, feature maps, pooling, famous CNNs and transfer learning

Next: **Book 4, Guessing Which One** — 'how much' is not 'which one'. Nova must choose: is that my name, or not? A neuron that answers with a probability, and a loss made for choices.